UNIVERSITÉ SORBONNE PARIS NORD

INSTITUT GALILÉE

Base de données avancées

TP2- Requêtes, dépendances fonctionnelles et normalisation

Fait par :

Khaoula Charef

Numéro Etudiant :

12509041

Formation

Ingénierie Informatique

Exercice 1

Question 1 : Afficher le nom du département qui a le budget le plus élevé

L'objectif de cette question est d'identifier le département dont le budget est maximal dans la relation `department`. Pour cela, on compare le budget de chaque département avec la valeur maximale des budgets calculée par une sous-requête.

```
SELECT dept_name
FROM department
WHERE budget IN
    (SELECT MAX(budget) FROM department);
```

La sous-requête `SELECT MAX(budget) FROM department` retourne le budget le plus élevé. La requête principale affiche ensuite le nom du ou des départements possédant ce budget.

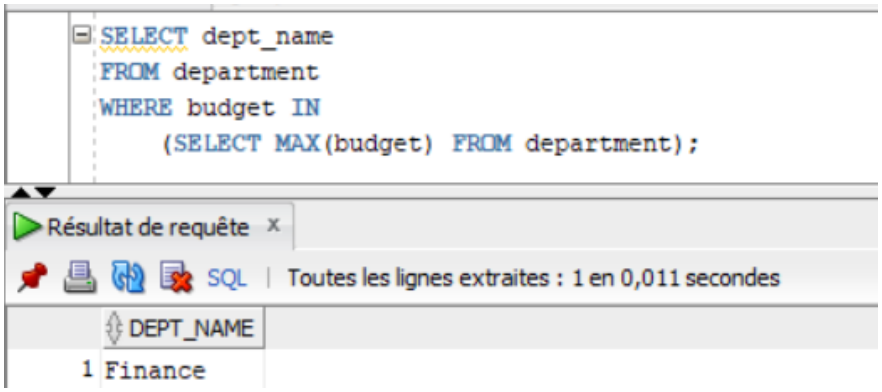


FIGURE 1 – Résultat de la question 1 dans SQL Developer

Question 2 : Afficher les salaires et les noms des enseignants qui gagnent plus que le salaire moyen

Dans cette question, on cherche les enseignants dont le salaire est strictement supérieur au salaire moyen de tous les enseignants. Le calcul de la moyenne est effectué dans une sous-requête.

```
SELECT A.name, A.salary
FROM teacher A
WHERE A.salary > (SELECT AVG(B.salary) FROM teacher B);
```

La sous-requête calcule le salaire moyen des enseignants de la table `teacher`. Ensuite, la requête principale sélectionne les enseignants dont le salaire dépasse cette moyenne.

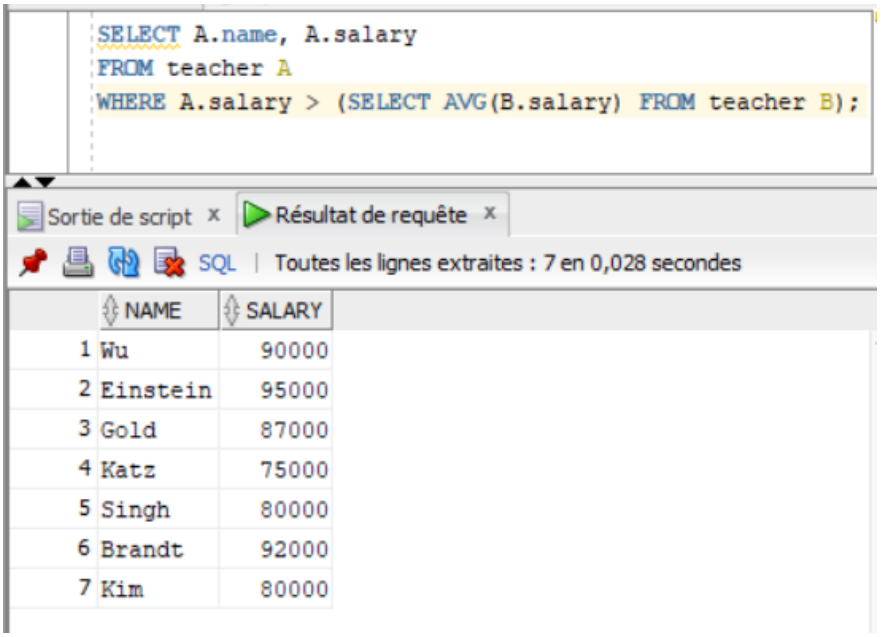


FIGURE 2 – Résultat de la question 2 dans SQL Developer

Question 3 : Pour chaque enseignant, afficher les étudiants qui ont suivi plus de deux cours dispensés par cet enseignant, en utilisant la clause `HAVING`

Cette requête relie les tables `teacher`, `student`, `takes` et `teaches` afin de déterminer, pour chaque enseignant, les étudiants ayant suivi plus de deux cours assurés par ce même enseignant.

```
SELECT teacher.name, student.name, COUNT(*)
FROM teacher, student, takes, teaches
WHERE teacher.id = teaches.id
  AND student.id = takes.id
  AND takes.course_id = teaches.course_id
  AND takes.sec_id = teaches.sec_id
  AND takes.semester = teaches.semester
  AND takes.year = teaches.year GROUP BY teacher.name, student.name HAVING COUNT(*) > 2;
```

La jointure permet de relier un étudiant à un enseignement donné, puis d'associer cet enseignement à l'enseignant concerné. La clause `GROUP BY` regroupe les résultats par enseignant et par étudiant. Enfin, la clause `HAVING` permet de ne conserver que les groupes dont le nombre de cours suivis est supérieur ou égal à 2.

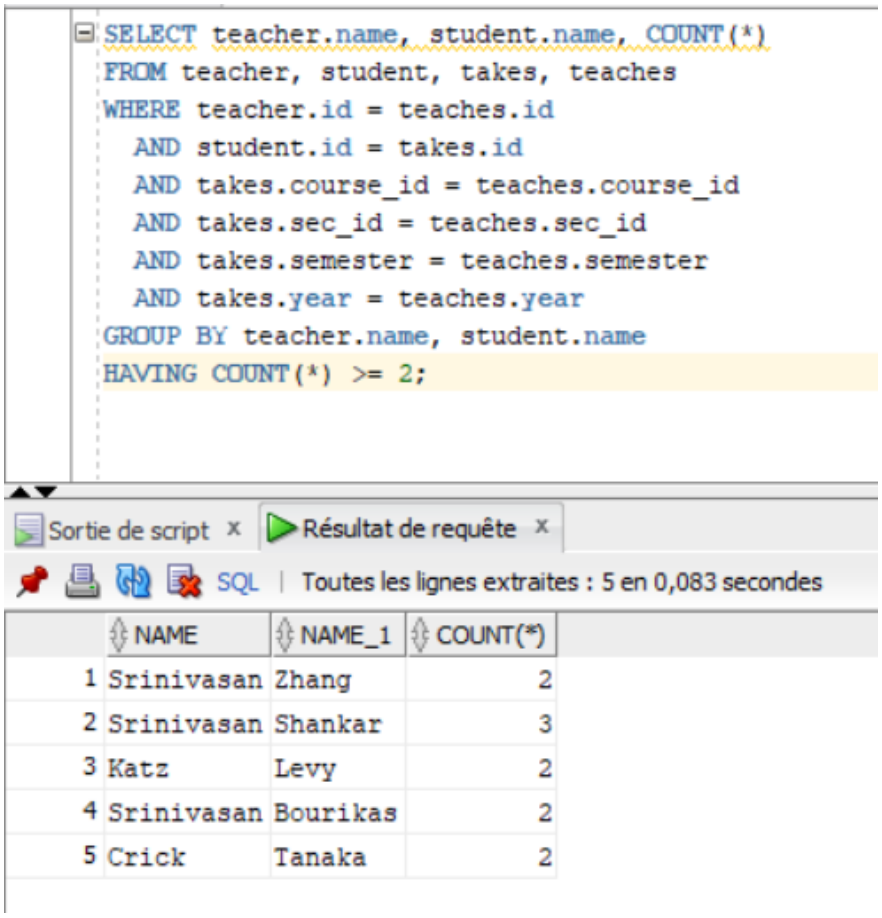


FIGURE 3 – Résultat de la question 3 dans SQL Developer

Question 4 : Pour chaque enseignant, afficher les étudiants qui ont suivi plus de deux cours dispensés par cet enseignant, sans utiliser la clause `HAVING`

Cette question reprend le même objectif que la précédente, mais sans utiliser la clause `HAVING`. On passe donc par une sous-requête dans laquelle on calcule d'abord le nombre total de cours suivis par chaque étudiant avec chaque enseignant.

```
SELECT T.teachername, T.studentname, T.totalcount
FROM (
    SELECT teacher.name AS teachername,
           student.name AS studentname,
           COUNT(*) AS totalcount
    FROM teacher, student, takes, teaches
    WHERE teacher.id = teaches.id
      AND student.id = takes.id
      AND takes.course_id = teaches.course_id
      AND takes.sec_id = teaches.sec_id
      AND takes.semester = teaches.semester
      AND takes.year = teaches.year
    GROUP BY teacher.name, student.name
) T
WHERE T.totalcount > 2
ORDER BY T.teachername;
```

La sous-requête calcule d'abord le nombre de cours suivis par étudiant pour chaque enseignant. Ensuite, la requête externe filtre les résultats pour ne garder que les cas où ce nombre est supérieur ou égal à 2.

```
SELECT T.teachername, T.studentname, T.totalcount
FROM (
    SELECT teacher.name AS teachername,
           student.name AS studentname,
           COUNT(*) AS totalcount
    FROM teacher, student, takes, teaches
    WHERE teacher.id = teaches.id
      AND student.id = takes.id
      AND takes.course_id = teaches.course_id
      AND takes.sec_id = teaches.sec_id
      AND takes.semester = teaches.semester
      AND takes.year = teaches.year
    GROUP BY teacher.name, student.name
) T
WHERE T.totalcount >= 2
ORDER BY T.teachername;
```

Sortie de script | Résultat de requête

SQL | Toutes les lignes extraites : 5 en 0,029 secondes

	TEACHERNAME	STUDENTNAME	TOTALCOUNT
1	Crick	Tanaka	2
2	Katz	Levy	2
3	Srinivasan	Zhang	2
4	Srinivasan	Bourikas	2
5	Srinivasan	Shankar	3

FIGURE 4 – Résultat de la question 4 dans SQL Developer

Question 5 : Afficher les identifiants et les noms des étudiants qui n'ont pas suivi de cours avant 2010

```
SELECT student.id, student.name
FROM student
MINUS
SELECT student.id, student.name
FROM student, takes
WHERE takes.id = student.id
  AND takes.year < 2010;
```

La première requête retourne tous les étudiants. La seconde retourne les étudiants ayant suivi au moins un cours avant une certaine année. L'opérateur `MINUS` permet alors d'obtenir uniquement les étudiants qui ne sont pas dans ce second ensemble.

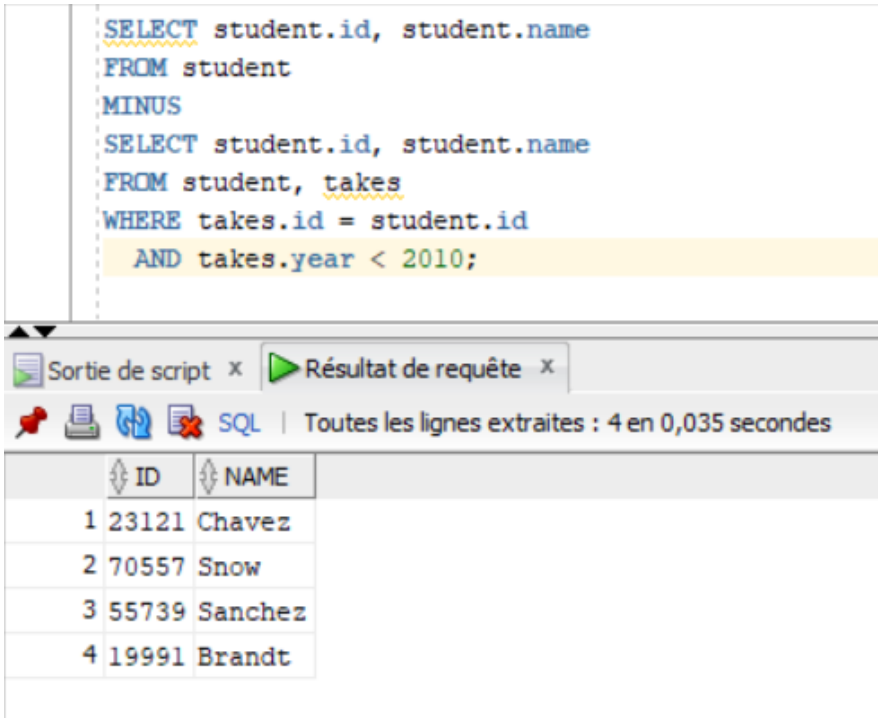


FIGURE 5 – Résultat de la question 5 dans SQL Developer

Question 6 : Afficher tous les enseignants dont les noms commencent par E

Le but ici est de sélectionner les enseignants dont le nom commence par la lettre `E`. Pour cela, on utilise l'opérateur `LIKE`.

```
SELECT *
FROM teacher
WHERE name LIKE 'E%';
```

L'expression `'E%'` signifie que le nom doit commencer par la lettre `E`, suivie de n'importe quelle suite de caractères.

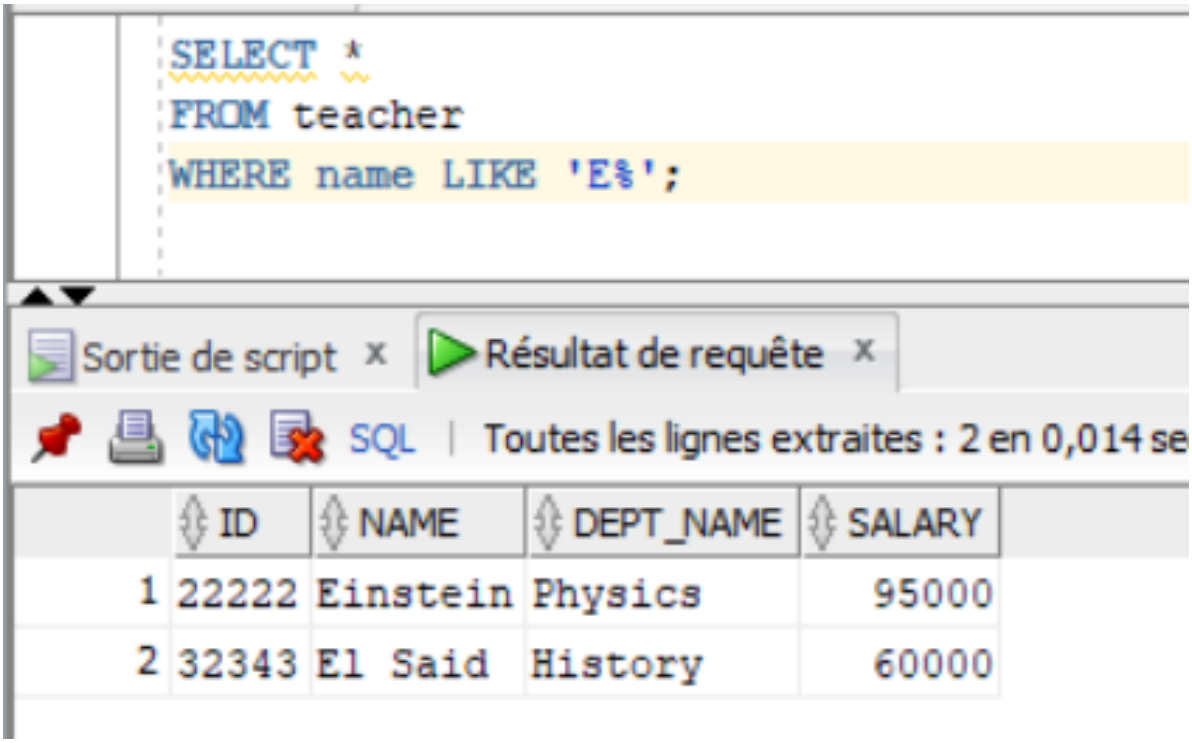


FIGURE 6 – Résultat de la question 6 dans SQL Developer

Question 7 : Afficher les salaires et les noms des enseignants qui perçoivent le quatrième salaire le plus élevé

```
SELECT name, salary
FROM teacher T1
WHERE 3 = (
    SELECT COUNT(DISTINCT T2.salary)
    FROM teacher T2
    WHERE T2.salary > T1.salary
);
```

La sous-requête compte, pour chaque enseignant, combien de salaires distincts sont supérieurs au sien. Si exactement trois salaires sont plus élevés, alors le salaire considéré est le quatrième plus élevé.

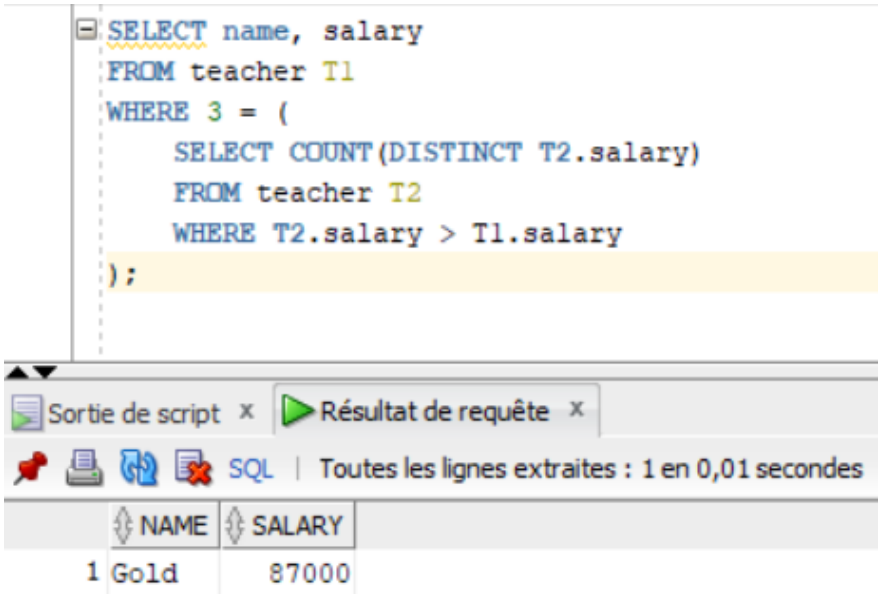


FIGURE 7 – Résultat de la question 7 dans SQL Developer

Question 8 : Afficher les noms et les salaires des trois enseignants qui perçoivent les salaires les moins élevés, les afficher par ordre décroissant

```
SELECT T1.name, T1.salary
FROM teacher T1
WHERE 2 >= (
    SELECT COUNT(DISTINCT T2.salary)
    FROM teacher T2
    WHERE T2.salary < T1.salary
)
ORDER BY T1.salary DESC;
```

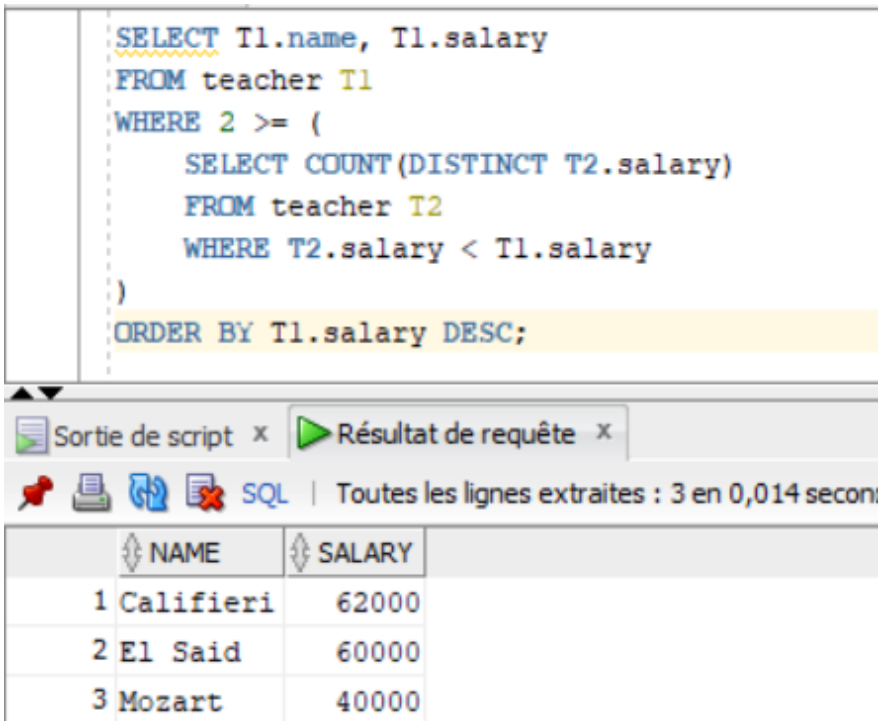


FIGURE 8 – Résultat de la question 8 dans SQL Developer

Question 9 : Afficher les noms des étudiants qui ont suivi un cours en automne 2009, en utilisant la clause `IN`

```
SELECT S.name
FROM student S
WHERE ('Fall', 2009) IN (
    SELECT semester, year
    FROM takes
    WHERE takes.id = S.id
);
```

La sous-requête retourne, pour chaque étudiant, les couples `(semester, year)` correspondant aux cours qu'il a suivis. La requête principale sélectionne alors les étudiants pour lesquels le couple `(Fall, 2009)` apparaît.

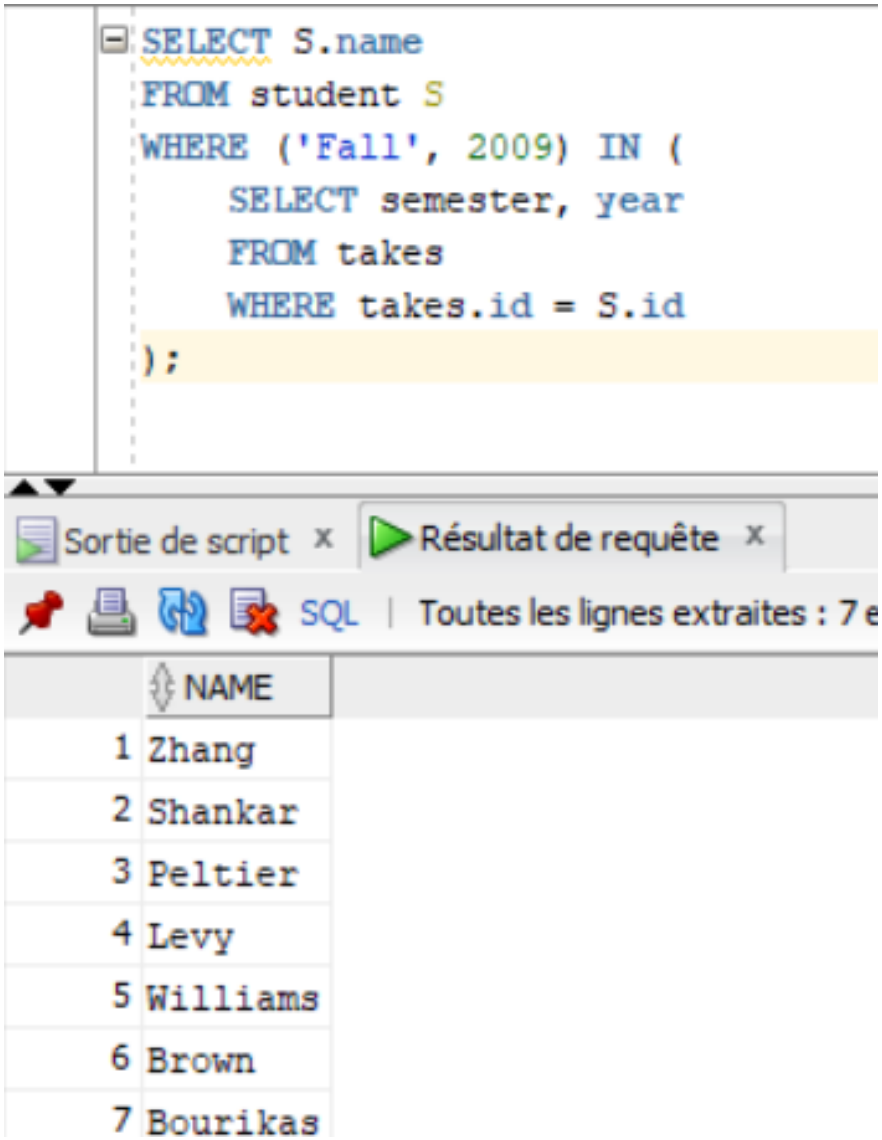


FIGURE 9 – Résultat de la question 9 dans SQL Developer

Question 10 : Afficher les noms des étudiants qui ont suivi un cours en automne 2009, en utilisant la clause SOME

Cette question est similaire à la précédente, mais on utilise cette fois la clause `SOME`. Elle permet de vérifier si le couple `(Fall, 2009)` est égal à au moins une des lignes retournées par la sous-requête.

```
WHERE ('Fall', 2009) = SOME (
    SELECT semester, year
    FROM takes
    WHERE takes.id = S.id
);
```

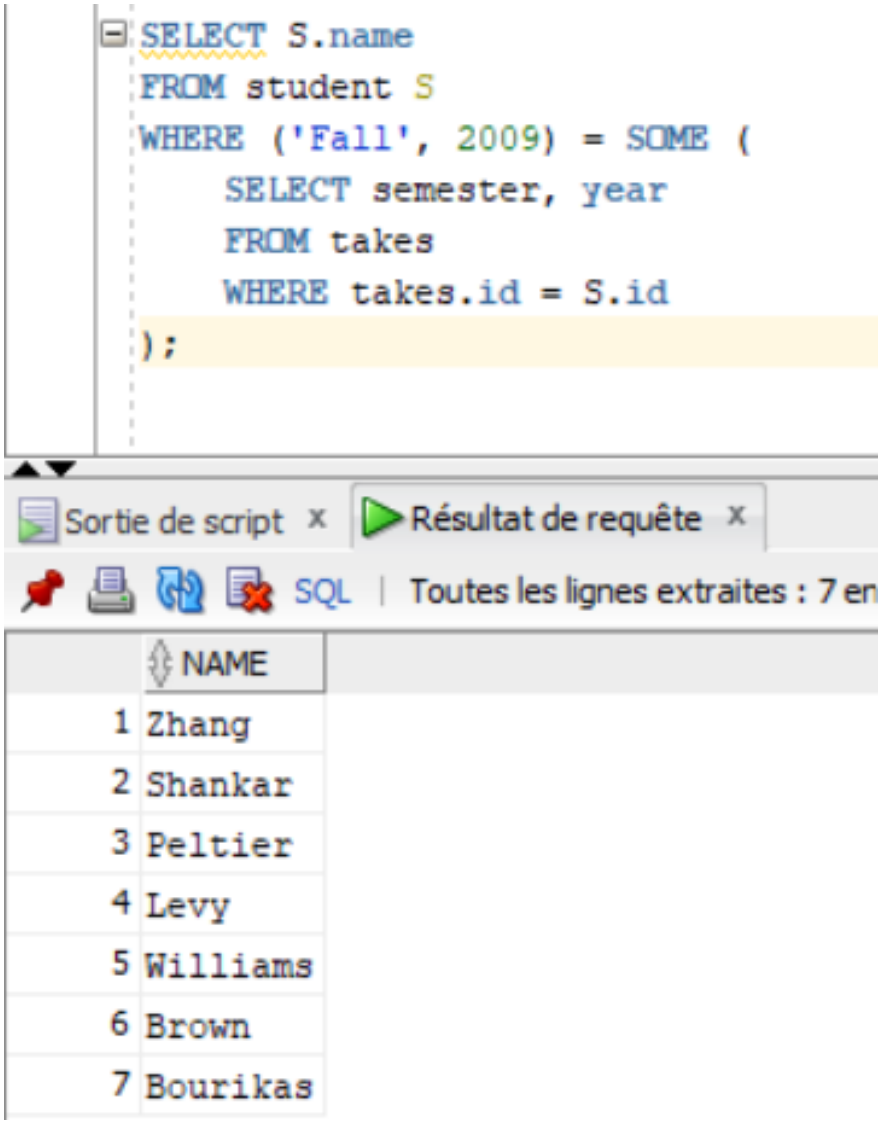


FIGURE 10 – Résultat de la question 10 dans SQL Developer

Question 11 : Afficher les noms des étudiants qui ont suivi un cours en automne 2009, en utilisant la jointure naturelle

Dans cette question, on utilise une `NATURAL INNER JOIN` entre les tables `takes` et `student`. Cette jointure naturelle relie automatiquement les colonnes communes aux deux tables, en particulier l'identifiant de l'étudiant.

```
SELECT DISTINCT student.name
FROM takes NATURAL INNER JOIN student
WHERE takes.semester = 'Fall' AND takes.year = 2009;
```

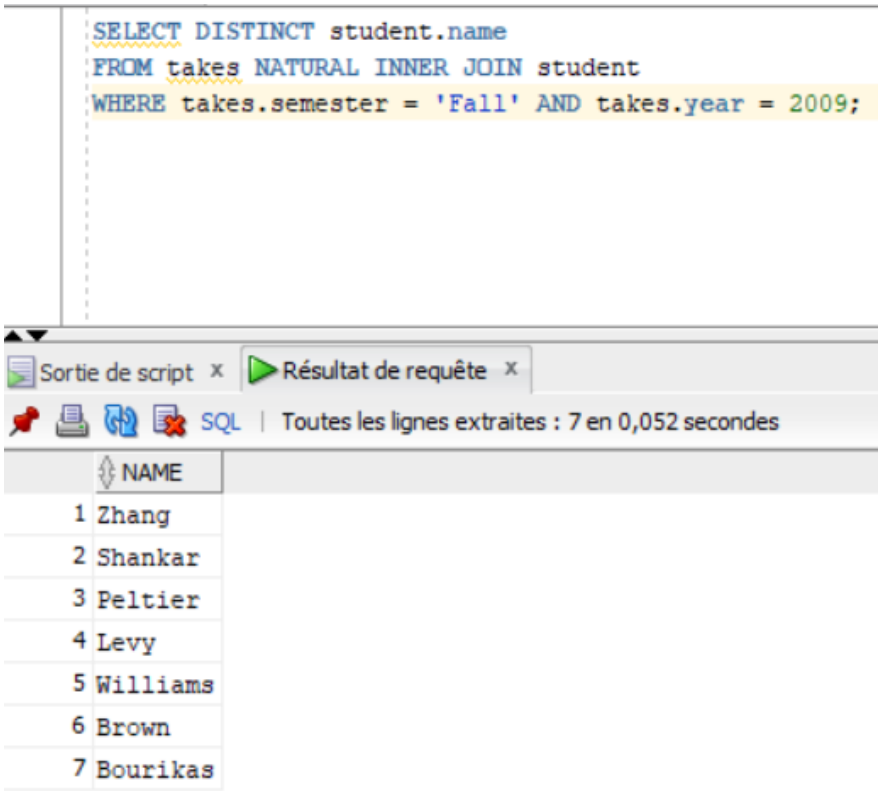


FIGURE 11 – Résultat de la question 11 dans SQL Developer

Question 12 : Afficher les noms des étudiants qui ont suivi un cours en automne 2009, en utilisant la clause EXISTS

Cette question reprend le même objectif que les précédentes, mais en utilisant cette fois la clause `EXISTS`. Cette clause permet de tester l'existence d'au moins une ligne satisfaisant certaines conditions dans la sous-requête.

```
SELECT name
FROM student
WHERE EXISTS (
    SELECT *
    FROM takes
    WHERE takes.id = student.id
      AND semester = 'Fall'
      AND year = 2009
);
```

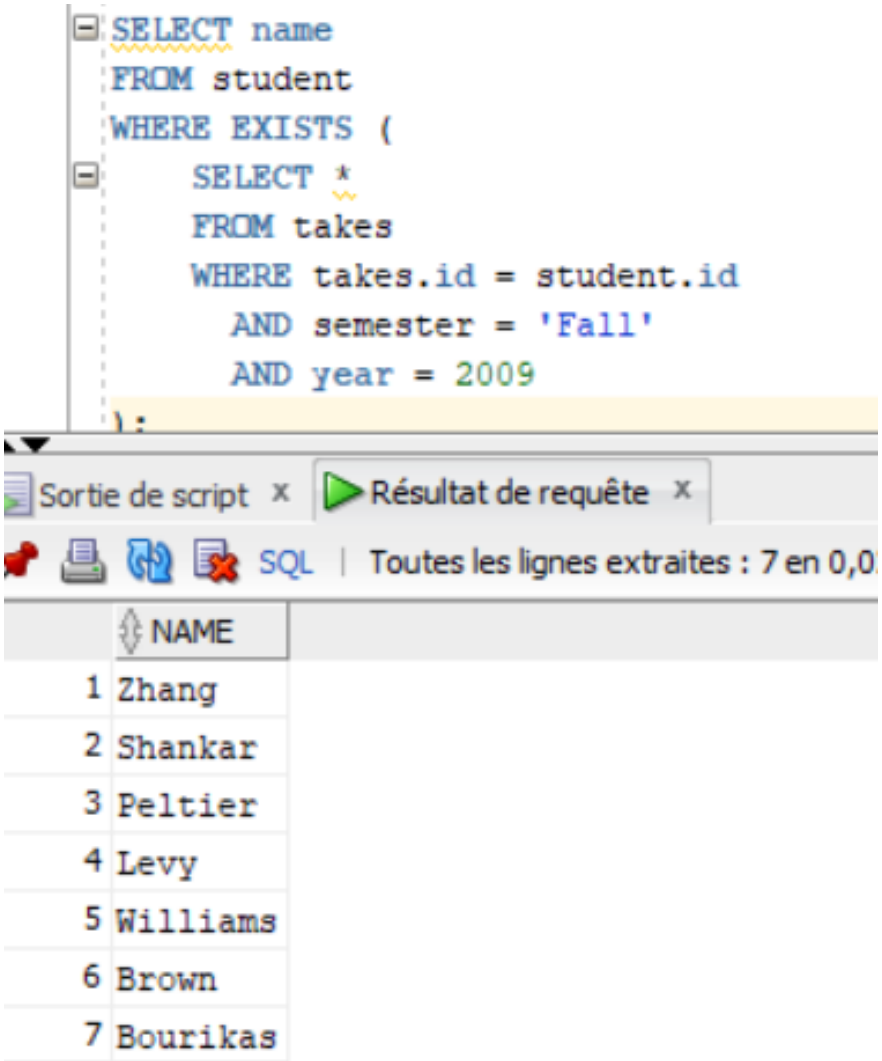


FIGURE 12 – Résultat de la question 12 dans SQL Developer

Question 13 : Afficher toutes les paires des étudiants qui ont suivi au moins un cours ensemble

L'objectif de cette question est d'identifier les paires d'étudiants ayant suivi au moins un même cours, dans la même section, le même semestre et la même année. Pour cela, on effectue une auto-jointure sur la table `takes`.

```
SELECT A.name, B.name
FROM (
    SELECT *
    FROM student NATURAL JOIN takes
) A,
(
    SELECT *
    FROM student NATURAL JOIN takes
) B
WHERE A.course_id = B.course_id
  AND A.sec_id = B.sec_id
  AND A.semester = B.semester
```

```
  AND A.year = B.year
  AND A.id <> B.id
  AND A.name < B.name
GROUP BY A.id, A.name, B.id, B.name
HAVING COUNT(*) >= 1;
```

```
SELECT A.name, B.name
FROM (
    SELECT *
    FROM student NATURAL JOIN takes
) A,
(
    SELECT *
    FROM student NATURAL JOIN takes
) B
WHERE A.course_id = B.course_id
  AND A.sec_id = B.sec_id
  AND A.semester = B.semester
```

Sortie de script | Résultat de requête

SQL | Toutes les lignes extraites : 15 en 0,059 secon

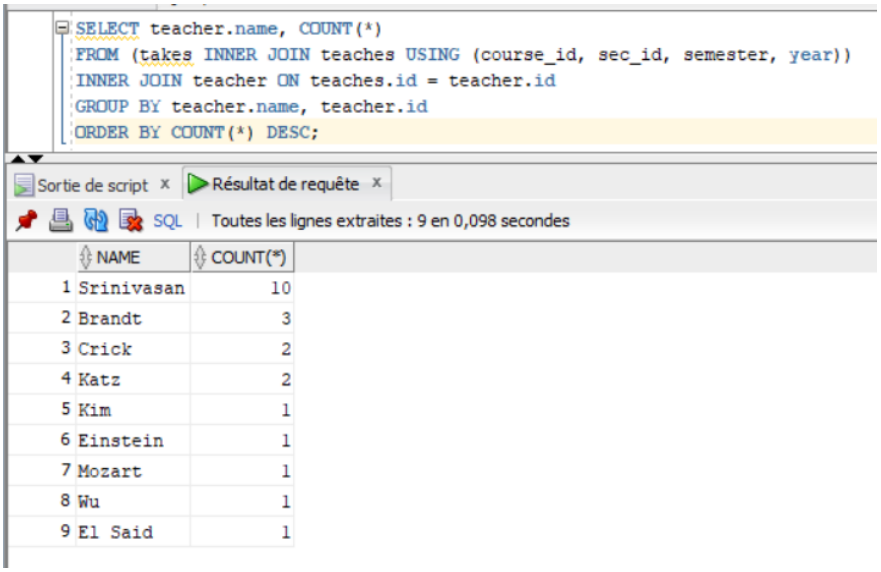
	NAME	NAME_1
1	Shankar	Williams
2	Shankar	Zhang
3	Levy	Shankar
4	Levy	Williams
5	Levy	Zhang
6	Williams	Zhang
7	Brown	Levy
8	Brown	Shankar
9	Brown	Williams
10	Brown	Zhang
11	Bourikas	Brown
12	Bourikas	Levy
13	Bourikas	Shankar
14	Bourikas	Williams
15	Bourikas	Zhang

Figure 13 – Résultat de la question 13 dans SQL Developer

Question 14 : Afficher pour chaque enseignant, qui a effectivement assuré un cours, le nombre total d'étudiants ayant suivi ses cours

Dans cette question, on cherche à déterminer, pour chaque enseignant ayant réellement assuré un cours, le nombre total d'étudiants qui ont suivi ses enseignements. Si un étudiant a suivi deux cours différents avec le même enseignant, il est compté deux fois. Le résultat est trié par ordre décroissant du nombre obtenu.

```
SELECT teacher.name, COUNT(*)
FROM (takes INNER JOIN teaches USING (course_id, sec_id, semester, year))
INNER JOIN teacher ON teaches.id = teacher.id
GROUP BY teacher.name, teacher.id
ORDER BY COUNT(*) DESC;
```



	NAME	COUNT(*)
1	Srinivasan	10
2	Brandt	3
3	Crick	2
4	Katz	2
5	Kim	1
6	Einstein	1
7	Mozart	1
8	Wu	1
9	El Said	1

FIGURE 14 – Résultat de la question 14 dans SQL Developer

Question 15 : Afficher pour chaque enseignant, même s'il n'a pas assuré de cours, le nombre total d'étudiants ayant suivi ses cours

Cette question est proche de la précédente, mais ici on veut également afficher les enseignants qui n'ont assuré aucun cours. Pour cela, on utilise une jointure externe droite avec la table `teacher`. Si un étudiant a suivi deux fois un cours avec le même enseignant, il est compté deux fois.

```
SELECT teacher.name, COUNT(course_id)
FROM (takes INNER JOIN teaches USING (course_id, sec_id, semester, year))
RIGHT OUTER JOIN teacher ON teaches.id = teacher.id
GROUP BY teacher.name, teacher.id
ORDER BY COUNT(course_id) DESC;
```

```
SELECT teacher.name, COUNT(course_id)
FROM (takes INNER JOIN teaches USING (course_id, sec_id, semester, year))
RIGHT OUTER JOIN teacher ON teaches.id = teacher.id
GROUP BY teacher.name, teacher.id
ORDER BY COUNT(course_id) DESC;
```

Sortie de script × | Résultat de requête ×

SQL | Toutes les lignes extraites : 12 en 0,051 secondes

	NAME	COUNT(COURSE_ID)
1	Srinivasan	10
2	Brandt	3
3	Katz	2
4	Crick	2
5	Einstein	1
6	Mozart	1
7	Kim	1
8	Wu	1
9	El Said	1
10	Gold	0
11	Califieri	0
12	Singh	0

FIGURE 15 – Résultat de la question 15 dans SQL Developer

Question 16 : Pour chaque enseignant, afficher le nombre total de grades A qu'il a attribué

L'objectif de cette question est de calculer, pour chaque enseignant, le nombre total de notes `A` attribuées dans les cours qu'il a assurés. Pour cela, on commence par sélectionner les inscriptions dont la note est égale à `A`, puis on les relie aux enseignants concernés.

```
WITH mytake(id, course_id, sec_id, semester, year, grade) AS (
    SELECT id, course_id, sec_id, semester, year, grade
    FROM takes
    WHERE grade = 'A'
)
SELECT teacher.name, COUNT(course_id)
FROM (mytake INNER JOIN teaches USING (course_id, sec_id, semester, year))
RIGHT OUTER JOIN teacher ON teaches.id = teacher.id
GROUP BY teacher.name, teacher.id
ORDER BY COUNT(course_id) DESC;
```

La clause `WITH` permet de définir une table temporaire contenant uniquement les inscriptions ayant obtenu la note `A`. On relie ensuite ces données à la table `teaches`, puis à la table `teacher`, afin d'obtenir le nombre de notes `A` attribuées par chaque enseignant.

```
WITH mytake(id, course_id, sec_id, semester, year, grade) AS (
    SELECT id, course_id, sec_id, semester, year, grade
    FROM takes
    WHERE grade = 'A'
)
SELECT teacher.name, COUNT(course_id)
FROM (mytakes INNER JOIN teaches USING (course_id, sec_id, semester, year))
RIGHT OUTER JOIN teacher ON teaches.id = teacher.id
GROUP BY teacher.name, teacher.id
ORDER BY COUNT(course_id) DESC;
```

Sortie de script | Résultat de requête

SQL | Toutes les lignes extraites : 12 en 0,051 secondes

	NAME	COUNT(COURSE_ID)
1	Srinivasan	10
2	Brandt	3
3	Katz	2
4	Crick	2
5	Einstein	1
6	Mozart	1
7	Kim	1
8	Wu	1
9	El Said	1
10	Gold	0
11	Califieri	0
12	Singh	0

FIGURE 16 – Résultat de la question 16 dans SQL Developer

Question 17 : Afficher toutes les paires enseignant–élève où un élève a suivi le cours de l'enseignant, ainsi que le nombre de fois

Dans cette question, on veut obtenir toutes les paires formées d'un enseignant et d'un étudiant lorsque l'étudiant a suivi un cours dispensé par cet enseignant. On souhaite également connaître le nombre de fois où cette situation s'est produite.

```
SELECT teacher.name, student.name, COUNT(*)
FROM (teacher NATURAL JOIN teaches)
INNER JOIN
(takes NATURAL JOIN student) USING (course_id, sec_id, semester, year)
GROUP BY teacher.name, student.name;
```

La première partie associe chaque enseignant aux cours qu'il a dispensés, tandis que la seconde associe chaque étudiant aux cours qu'il a suivis. La jointure entre les deux ensembles permet d'identifier les couples enseignant–étudiant correspondants, puis de compter le nombre d'occurrences pour chaque paire.

```
SELECT teacher.name, student.name, COUNT(*)
FROM (teacher NATURAL JOIN teaches)
INNER JOIN
(takes NATURAL JOIN student) USING (course_id, sec_id, semester, year)
GROUP BY teacher.name, student.name;
```

Sortie de script × | Résultat de requête ×

SQL | Toutes les lignes extraites : 16 en 0,121 secondes

	NAME	NAME_1	COUNT(*)
1	Srinivasan	Zhang	2
2	Srinivasan	Shankar	3
3	Brandt	Shankar	1
4	El Said	Brandt	1
5	Wu	Chavez	1
6	Einstein	Peltier	1
7	Srinivasan	Levy	1
8	Katz	Levy	2
9	Srinivasan	Williams	1
10	Brandt	Williams	1
11	Mozart	Sanchez	1
12	Srinivasan	Brown	1
13	Brandt	Brown	1
14	Kim	Aoi	1
15	Srinivasan	Bourikas	2
16	Crick	Tanaka	2

FIGURE 17 – Résultat de la question 17 dans SQL Developer

Question 18 : Afficher toutes les paires enseignant–élève où un élève a suivi au moins deux cours dispensés par le même enseignant

Dans cette question, on cherche les paires enseignant–élève pour lesquelles l'étudiant a suivi au moins deux cours dispensés par cet enseignant. On calcule d'abord le nombre total de cours suivis pour chaque paire, puis on filtre les résultats afin de ne conserver que les cas où ce nombre est supérieur ou égal à 2.

```sql
SELECT mytable.tn, mytable.sn
FROM (
    SELECT teacher.name AS tn,
           student.name AS sn,
           COUNT(*) AS tc
    FROM (teacher NATURAL JOIN teaches)
    INNER JOIN
    (takes NATURAL JOIN student) USING (course_id, sec_id, semester, year)
    GROUP BY teacher.name, student.name
) mytable
WHERE tc >= 2;
```

La sous-requête construit d'abord l'ensemble des paires enseignant–étudiant et calcule, pour chacune d'elles, le nombre total de cours suivis ensemble. La requête externe ne conserve ensuite que les paires dont ce nombre est supérieur ou égal à 2.

```sql
SELECT mytable.tn, mytable.sn
FROM (
    SELECT teacher.name AS tn,
           student.name AS sn,
           COUNT(*) AS tc
    FROM (teacher NATURAL JOIN teaches)
    INNER JOIN
    (takes NATURAL JOIN student) USING (course_id, sec_id, semester, year)
    GROUP BY teacher.name, student.name
) mytable
WHERE tc >= 2;
```

Sortie de script | Résultat de requête

SQL | Toutes les lignes extraites : 5 en 0,026 secondes

	TN	SN
1	Srinivasan	Zhang
2	Srinivasan	Shankar
3	Katz	Levy
4	Srinivasan	Bourikas
5	Crick	Tanaka

FIGURE 18 – Résultat de la question 18 dans SQL Developer

Exercice 2

Dans cet exercice, on étudie plusieurs schémas de relation accompagnés de leurs dépendances fonctionnelles. Pour chaque cas, on détermine d'abord la clé candidate, puis on identifie la forme normale la plus avancée atteinte par la relation. Lorsque la relation ne satisfait pas la forme normale suivante, on propose une décomposition permettant d'obtenir une structure mieux normalisée.

1. $R(A, B, C)$ avec $F = \{A \to B,\ B \to C\}$

On commence par déterminer la clé candidate de la relation.

À partir des dépendances fonctionnelles :

$$A \to B \quad \text{et} \quad B \to C$$

on déduit par transitivité que :

$$A \to C$$

Ainsi :

$$A^+ = \{A, B, C\}$$

Donc A **est une clé candidate** de la relation.

L'attribut premier est donc A, tandis que B et C sont des attributs non premiers.

La relation est en **deuxième forme normale (2FN)**, car la clé est formée d'un seul attribut, donc aucune dépendance partielle n'est possible.

En revanche, la relation n'est pas en **troisième forme normale (3FN)**, car la dépendance :

$$B \to C$$

montre qu'un attribut non-clé détermine un autre attribut non-clé. Il existe donc une dépendance transitive :

$$A \to B \to C$$

La forme normale la plus avancée atteinte est donc la **2FN**.

Pour obtenir une relation en 3FN, on peut proposer la décomposition suivante :

$$R_1(B, C)$$

$$R_2(A, B)$$

Dans cette décomposition, la dépendance transitive est supprimée, et les deux relations obtenues sont en 3FN.

2. $R(A, B, C)$ avec $F = \{A \to C,\ A \to B\}$

On calcule d'abord la fermeture de l'attribut A.

D'après les dépendances fonctionnelles :

$$A \to B \quad \text{et} \quad A \to C$$

on obtient directement :

$$A^+ = \{A, B, C\}$$

Ainsi, A **est une clé candidate** de la relation.

Comme la clé est composée d'un seul attribut, la relation est automatiquement en **2FN**. De plus, toutes les dépendances fonctionnelles non triviales ont pour partie gauche la clé candidate A.

La relation satisfait donc la **3FN**, et même la **forme normale de Boyce-Codd (BCNF)**, car dans chaque dépendance fonctionnelle non triviale, le déterminant est une super-clé.

La forme normale la plus avancée atteinte est donc la **BCNF**.

Aucune décomposition n'est nécessaire.

3. $R(A, B, C)$ avec $F = \{A, B \to C;\ C \to B\}$

On cherche d'abord les clés candidates de la relation.

À partir de la dépendance :

$$AB \to C$$

on obtient :

$$(AB)^+ = \{A, B, C\}$$

Donc AB **est une clé candidate**.

Par ailleurs, comme :

$$C \to B$$

on peut aussi écrire que :

$$(AC)^+ = \{A, C\} \cup \{B\} = \{A, B, C\}$$

Donc AC **est également une clé candidate**.

Les attributs A, B et C sont donc tous des **attributs premiers**, car chacun appartient à au moins une clé candidate.

La relation est en **3FN** :

- la dépendance $AB \to C$ a pour partie gauche une clé candidate ;
- la dépendance $C \to B$ a pour partie droite un attribut premier.

En revanche, la relation n'est pas en **BCNF**, car dans la dépendance :

$$C \to B$$

la partie gauche C n'est pas une super-clé.

La forme normale la plus avancée atteinte est donc la **3FN**.

Pour obtenir une décomposition en BCNF, on peut proposer :

$$R_1(C, B)$$

$$R_2(A, C)$$

Cette décomposition élimine la violation de la BCNF.

Exercice 3

1. Déduction de dépendances fonctionnelles à l'aide des axiomes d'Armstrong

On considère la relation $R(A, B, C, D, E)$ munie de l'ensemble de dépendances fonctionnelles :

$$F = \{A \to B, C;\ C, D \to E;\ B \to D;\ E \to A\}$$

À partir de ces dépendances et en utilisant les axiomes d'Armstrong (réflexivité, augmentation et transitivité), on peut déduire plusieurs autres dépendances fonctionnelles.

Les dépendances données directement sont :

$$A \to B$$

$$A \to C$$

$$CD \to E$$

$$B \to D$$

$$E \to A$$

On peut ensuite déduire les dépendances suivantes :

$$A \to D$$

par transitivité à partir de $A \to B$ et $B \to D$.

$$E \to B$$

par transitivité à partir de $E \to A$ et $A \to B$.

$$E \to C$$

par transitivité à partir de $E \to A$ et $A \to C$.

$$E \to D$$

par transitivité à partir de $E \to B$ et $B \to D$.

$$A \to BC$$

par union à partir de $A \to B$ et $A \to C$.

$$E \to BC$$

par union à partir de $E \to B$ et $E \to C$.

$$E \to ABC$$

car $E \to A$, $E \to B$ et $E \to C$.

$$E \to ABCD$$

car $E \to A$, $E \to B$, $E \to C$ et $E \to D$.

$$A \to BD$$

car $A \to B$ et $A \to D$.

$$A \to CD$$

car $A \to C$ et $A \to D$.

$$A \to E$$

car $A \to C$ et $A \to D$, donc $A \to CD$, puis avec $CD \to E$, on obtient $A \to E$.

$$A \to ABCDE$$

car à partir de A, on obtient B, C, D et E.

$$E \to E$$

par réflexivité.

$$AB \to DB$$

par augmentation à partir de $B \to D$.

$$ACD \to AE$$

par augmentation à partir de $CD \to E$.

$$BE \to DE$$

par augmentation à partir de $B \to D$.

$$CE \to CA$$

par augmentation à partir de $E \to A$.

On obtient ainsi largement plus de 16 dépendances fonctionnelles déduites de l'ensemble initial.

2. Étude du schéma $R(A, B, C, D, E, F)$

On considère la relation :

$$R(A, B, C, D, E, F)$$

munie de l'ensemble de dépendances fonctionnelles :

$$F = \{A \to B, C, D;\ B, C \to D, E;\ B \to D;\ D \to A\}$$

(a) Calcul de la fermeture de B et de l'ensemble $\{A, B\}$

Commençons par calculer la fermeture de l'attribut B.

À partir de :

$$B \to D$$

on obtient D.

Puis, comme :

$$D \to A$$

on obtient aussi A.

Ensuite, avec :

$$A \to B, C, D$$

on obtient C.

Enfin, comme on dispose maintenant de B et C, on applique :

$$BC \to D, E$$

ce qui donne E.

Ainsi :

$$B^+ = \{A, B, C, D, E\}$$

L'attribut F ne peut pas être déduit, donc la fermeture de B est :

$$B^+ = \{A, B, C, D, E\}$$

Calculons maintenant la fermeture de l'ensemble $\{A, B\}$.
À partir de :

$$A \to B, C, D$$

on obtient C et D.
Puis, comme B et C sont présents, on applique :

$$BC \to D, E$$

et on obtient E.
Ainsi :

$$\{A, B\}^+ = \{A, B, C, D, E\}$$

Là encore, on ne peut pas obtenir F, donc :

$$\{A, B\}^+ = \{A, B, C, D, E\}$$

(b) Montrer que $\{A, F\}$ est une super-clé

Considérons l'ensemble d'attributs $\{A, F\}$.
À partir de la dépendance :

$$A \to B, C, D$$

on obtient les attributs B, C et D.
Comme on possède désormais B et C, on peut utiliser :

$$BC \to D, E$$

ce qui permet d'obtenir aussi E.
Ainsi, à partir de $\{A, F\}$, on obtient :

$$A, B, C, D, E, F$$

Donc :

$$(AF)^+ = \{A, B, C, D, E, F\}$$

Par conséquent, $\{A, F\}$ **est une super-clé** de la relation.

(c) La relation est-elle en BCNF ? Si non, la décomposer

On considère la relation :

$$R(A, B, C, D, E, F)$$

avec l'ensemble de dépendances fonctionnelles :

$$F = \{A \to B, C, D;\ B, C \to D, E;\ B \to D;\ D \to A\}$$

Pour vérifier si la relation est en **BCNF**, il faut que, pour toute dépendance fonctionnelle non triviale

$$X \to Y$$

le déterminant X soit une super-clé.
Or, on a la dépendance :

$$A \to B, C, D$$

Calculons la fermeture de A :

$$A^+ = \{A, B, C, D, E\}$$

En effet :

— à partir de $A \to B, C, D$, on obtient B, C et D ;
— puis avec $BC \to D, E$, on obtient E.

Cependant, on n'obtient pas F, donc :

$$A^+ \neq \{A, B, C, D, E, F\}$$

Ainsi, A **n'est pas une super-clé** de la relation.

La dépendance

$$A \to B, C, D$$

viole donc la condition de la BCNF.

Par conséquent, **la relation R n'est pas en BCNF**.

Pour obtenir une décomposition en BCNF, on peut décomposer R selon cette dépendance en :

$$R_1(A, B, C, D, E)$$
$$R_2(A, F)$$

Dans la relation $R_1(A, B, C, D, E)$, on a :
— $A \to B, C, D, E$;
— $B \to D$;
— $D \to A$;
— $BC \to D, E$.

On remarque que dans R_1 :
— A détermine tous les attributs de R_1, donc A est une super-clé ;
— $D \to A$ et $A \to B, C, D, E$, donc D détermine aussi tous les attributs de R_1 ;
— $B \to D \to A \to B, C, D, E$, donc B est également une super-clé.

Ainsi, R_1 **est en BCNF**.

La relation $R_2(A, F)$ ne contient que deux attributs et aucune dépendance fonctionnelle non triviale ne viole la BCNF. Donc R_2 **est aussi en BCNF**.

En conclusion, la décomposition :

$$R_1(A, B, C, D, E) \quad \text{et} \quad R_2(A, F)$$

permet d'obtenir une décomposition en **BCNF**.

3. Étude de décompositions

(a) Décomposition de $R(A, B, C, D, E)$ en $R_1(A, B, C)$ et $R_2(A, D, E)$

On considère la décomposition :

$$R(A, B, C, D, E) \longrightarrow R_1(A, B, C) \text{ et } R_2(A, D, E)$$

L'intersection des deux relations est :

$$R_1 \cap R_2 = \{A\}$$

Cette décomposition est **sans perte d'information**, car les deux relations partagent l'attribut A, qui joue ici le rôle d'attribut de raccordement entre les deux projections.

En effet, lors de la jonction naturelle :

$$R_1 \bowtie R_2$$

les tuples de R_1 et de R_2 sont recombinés à partir de la valeur commune de A, ce qui permet de reconstruire la relation initiale sans créer de tuples parasites.

Ainsi, on a :

$$R = R_1 \bowtie R_2$$

La décomposition :

$$R(A, B, C, D, E) \to R_1(A, B, C),\ R_2(A, D, E)$$

est donc **sans perte d'information**.

(b) Décomposition de $R(A, B, C, D, E)$ en $R_1(A, B, C)$ et $R_2(C, D, E)$

On considère maintenant la décomposition :

$$R(A, B, C, D, E) \longrightarrow R_1(A, B, C) \text{ et } R_2(C, D, E)$$

L'intersection des deux relations est :

$$R_1 \cap R_2 = \{C\}$$

Cette décomposition est considérée comme **avec perte d'information**, car la jonction naturelle :

$$R_1 \bowtie R_2$$

peut produire des tuples parasites, c'est-à-dire des combinaisons de valeurs qui n'existaient pas dans la relation initiale.

En effet, le simple fait d'avoir l'attribut commun C ne garantit pas que la reconstruction de la relation initiale soit exacte. Lors de la recombinaison, plusieurs tuples de R_1 et de R_2 peuvent être associés à tort dès qu'ils possèdent la même valeur de C.

Ainsi, la jonction naturelle des deux projections ne permet pas toujours de retrouver exactement la relation de départ.

La décomposition de $R(A, B, C, D, E)$ en $R_1(A, B, C)$ et $R_2(C, D, E)$ est avec perte d'information, car la jonction naturelle se fait uniquement sur l'attribut C. Si plusieurs tuples possèdent la même valeur de C, la jointure peut produire des tuples parasites, c'est-à-dire des tuples qui n'existaient pas dans la relation initiale. La reconstruction de R n'est donc pas exacte.

Exercice 4

Dans cet exercice, on utilise python comme langage de programmation pour manipuler des relations et des dépendances fonctionnelles. Les structures de données choisies représentent :

- une relation par un ensemble de chaînes de caractères ;
- un ensemble de relations par une liste d'ensembles ;
- un ensemble de dépendances fonctionnelles par une liste de paires d'ensembles.

Par exemple, les deux relations suivantes :

$$\{A, B, C, G, H, I\} \quad \text{et} \quad \{X, Y\}$$

peuvent être représentées par :

```
myrelations = [
    {'A', 'B', 'C', 'G', 'H', 'I'},
    {'X', 'Y'}
]
```

De même, l'ensemble des dépendances fonctionnelles

$$A \to B, \quad A \to C, \quad CG \to H, \quad CG \to I, \quad B \to H$$

peut être représenté par :

```
mydependencies = [
    [ {'A'}, {'B'} ], # A -> B
    [ {'A'}, {'C'} ], # A -> C
    [ {'C','G'}, {'H'} ], # CG -> H
    [ {'C','G'}, {'I'} ], # CG -> I
    [ {'B'}, {'H'} ] # B -> H
]
```

Question 1 : Écrire une procédure qui prend en paramètre une liste de dépendances fonctionnelles et les affiche

La procédure suivante prend en paramètre une liste de dépendances fonctionnelles et affiche, pour chacune d'elles, la partie gauche et la partie droite sous la forme d'une implication.

```
def printDependencies(F: "list of dependencies"):

    for alpha,beta in F:
        print("\t", alpha, " --> ", beta)
```

Question 2 : Écrire une procédure qui prend en paramètre un ensemble de relations T et les affiche

La procédure suivante permet d'afficher l'ensemble des relations contenues dans la liste T.

```
def printRelations(T: "list of relations"):

    for R in T:
        print("\t", R)
```

Question 3 : Écrire une fonction qui, prenant en paramètre un ensemble `inputset`, renvoie tous ses sous-ensembles

On souhaite ici générer tous les sous-ensembles d'un ensemble donné. On utilise pour cela le module `itertools`, qui fournit la fonction `combinations`.

```
import itertools

def powerSet(inputset: "set"):
    result = []
    for r in range(1, len(inputset)+1):
        result += list(map(set, itertools.combinations(inputset, r)))
    return result
```

Cette fonction parcourt toutes les tailles possibles de sous-ensembles, de 1 jusqu'à la taille de l'ensemble initial, puis construit la liste de toutes les combinaisons correspondantes.

Question 4 : Écrire une fonction qui retourne la fermeture d'un ensemble d'attributs K

On souhaite écrire une fonction qui, à partir d'un ensemble de dépendances fonctionnelles F et d'un ensemble d'attributs K, calcule la fermeture de K, c'est-à-dire l'ensemble de tous les attributs que l'on peut déduire à partir de K.

```
def computeAttributeClosure(F: "list of dependencies", K: "set of attributes"):
    K_plus, size = set(K), 0
    while size != len(K_plus):
        size = len(K_plus)
        for alpha, beta in F:
            if alpha.issubset(K_plus):
                K_plus.update(beta)
    return K_plus
```

La fonction commence par initialiser la fermeture avec l'ensemble K lui-même. Ensuite, tant que de nouveaux attributs peuvent être ajoutés, elle parcourt toutes les dépendances fonctionnelles de F. Si la partie gauche d'une dépendance est incluse dans la fermeture courante, alors la partie droite est ajoutée.

Question 5 : Écrire une fonction qui retourne la clôture de F

La clôture de F, notée F^+, est l'ensemble de toutes les dépendances fonctionnelles que l'on peut déduire à partir de F. La fonction suivante calcule cette clôture en considérant tous les sous-ensembles possibles des attributs de la relation.

```
def computeDependenciesClosure(F: "list of dependencies"):
    R = set()
    for alpha, beta in F:
        R.update(alpha | beta)

    F_plus = []
    for K in powerSet(R):
        for beta in powerSet(computeAttributeClosure(F, K)):
            F_plus.append([K, beta])

    return F_plus
```

La fonction commence par construire l'ensemble total des attributs apparaissant dans les dépendances fonctionnelles. Ensuite, pour chaque sous-ensemble K de cet ensemble, elle calcule la fermeture de K, puis génère toutes les dépendances de la forme $K \to \beta$.

Question 6 : Écrire une fonction qui vérifie si une dépendance fonctionnelle est vraie

On veut ici déterminer si, pour un ensemble de dépendances fonctionnelles F, une dépendance fonctionnelle $\alpha \to \beta$ est déductible. Pour cela, il suffit de vérifier si tous les attributs de β appartiennent à la fermeture de α.

```
def isDependency(F: "list of dependencies", alpha: "set of attributes", beta: "set of attributes"):
    return beta.issubset(computeAttributeClosure(F, alpha))
```

Cette fonction retourne `True` si la dépendance fonctionnelle $\alpha \to \beta$ est vérifiée, et `False` sinon.

Question 7 : Écrire une fonction qui vérifie si un ensemble d'attributs K est une super-clé

On cherche ici à déterminer si un ensemble d'attributs K constitue une super-clé d'une relation R. Pour cela, il suffit de vérifier si la fermeture de K contient tous les attributs de la relation R.

```
def isSuperKey(F: "list of dependencies", R: "set defining a relation", K: "set of attributes"):
    return R.issubset(computeAttributeClosure(F, K))
```

Cette fonction retourne `True` si K est une super-clé de R, et `False` sinon.

Question 8 : Écrire une fonction qui vérifie si un ensemble d'attributs K est une clé candidate

Une clé candidate est une super-clé minimale. La fonction suivante vérifie d'abord si K est une super-clé, puis teste si aucun sous-ensemble strict de K n'est encore une super-clé.

```
def isCandidateKey(F: "list of dependencies", R: "set defining a relation", K: "set of attributes")
    :
    if not isSuperKey(F, R, K):
        return False

    for A in K:
        _K1 = set(K)
```

```
        _K1.discard(A)
        if isSuperKey(F, R, _K1):
            return False

    return True
```

La fonction retourne `True` si K est une clé candidate, c'est-à-dire une super-clé minimale, et `False` sinon.

Question 9 : Écrire une fonction qui retourne la liste de toutes les clés candidates

On veut maintenant calculer l'ensemble de toutes les clés candidates d'une relation R. La fonction suivante parcourt tous les sous-ensembles de R et ne conserve que ceux qui sont des clés candidates.

```
def computeAllCandidateKeys(F: "list of dependencies", R: "set defining a relation"):
    result = []
    for K in powerSet(R):
        if isCandidateKey(F, R, K):
            result.append(K)

    return result
```

Cette fonction renvoie donc la liste complète de toutes les clés candidates de la relation.

Question 10 : Écrire une fonction qui retourne la liste de toutes les super-clés

Enfin, on souhaite obtenir toutes les super-clés de la relation R. La fonction suivante teste chaque sous-ensemble de R et ajoute à la liste des résultats ceux qui vérifient la propriété de super-clé.

```
def computeAllSuperKeys(F: "list of dependencies", R: "set defining a relation"):
    result = []
    for K in powerSet(R):
        if isSuperKey(F, R, K):
            result.append(K)

    return result
```

Cette fonction retourne la liste de tous les ensembles d'attributs qui déterminent entièrement la relation.

Question 11 : Écrire une fonction qui retourne une clé candidate

On souhaite ici calculer une clé candidate d'une relation R à partir d'un ensemble de dépendances fonctionnelles F. L'idée est de partir de l'ensemble complet des attributs de R, puis de retirer progressivement les attributs inutiles tout en conservant la propriété de clé candidate.

```
def computeOneCandidateKey(F: "list of dependencies", R: "set defining a relation"):
    K = set(R)
    while not isCandidateKey(F, R, K):
        for A in K:
            if isSuperKey(F, R, K.difference({A})):
                K.remove(A)
                break

    return K
```

Cette fonction retourne une clé candidate obtenue en supprimant les attributs non nécessaires à partir de la relation complète.

Question 12 : Écrire une fonction qui retourne vrai si la relation est en BCNF

La BCNF impose que, pour toute dépendance fonctionnelle non triviale $X \to Y$, l'ensemble X soit une super-clé. La fonction suivante vérifie cette propriété sur tous les sous-ensembles d'attributs de la relation.

```
def isBCNFRelation(F: "list of dependencies", R: "set defining a relation"):
    for X in powerSet(R):
        X_plus = computeAttributeClosure(F, X)
        Y = X_plus.difference(X)
        if not R.issubset(X_plus) and not Y.isdisjoint(R):
            return False, [X, Y & R]

    return True, [{}, {}]
```

Si la relation n'est pas en BCNF, la fonction retourne `False` ainsi qu'une dépendance fonctionnelle qui viole cette forme normale. Sinon, elle retourne `True`.

Question 13 : Écrire une fonction qui vérifie si le schéma défini par ces relations est en BCNF

On considère maintenant un schéma constitué de plusieurs relations. La fonction suivante vérifie si chacune de ces relations est en BCNF.

```
def isBCNFRelations(F: "list of dependencies", T: "list of relations"):
    for R in T:
        if isBCNFRelation(F, R) == False:
            return False, R

    return True, {}
```

La fonction parcourt toutes les relations de l'ensemble T. Si l'une d'elles n'est pas en BCNF, elle retourne `False` ainsi que la relation concernée. Sinon, elle retourne `True`.

Question 14 : Écrire une fonction qui implémente l'algorithme de décomposition en BCNF

Enfin, on veut implémenter l'algorithme de décomposition en BCNF. La fonction suivante applique itérativement la décomposition sur les relations qui ne satisfont pas la BCNF, jusqu'à obtenir un schéma entièrement en BCNF.

```
def computeBCNFDecomposition(F: "list of dependencies", T: "list of relations"):
    OUT = list(T)
    while size != len(OUT):
        size = len(OUT)
        for R in OUT:
            isR_BCNF, [alpha, beta] = isBCNFRelation(F, R)
            if isR_BCNF == False:
                if alpha | beta not in OUT:
                    OUT.append(alpha | beta)
                if R.difference(beta) not in OUT:
                    OUT.append(R.difference(beta))
                OUT.remove(R)
                break

    return OUT
```

Cette fonction prend comme entrée un ensemble de dépendances fonctionnelles F et un ensemble de relations T, puis retourne une décomposition de ces relations en BCNF.